

Rayce Ramsay (1009734888)
Ali Shabani (1008838652)
Terry Tian (1007663468)
Grant Hamblin (1009860447)
Hardik Shah (1009453690)

CSC490 A4 - Creating an RL LLM as a judge

Due: Nov 5th, 11pm with a Github PR + Quercus Submission

The goal of this assignment is to build an LLM as a judge that can evaluate metrics you care about.

1 Part One - Background research on GRPO (20 marks)

One of the major contributions of the Deepseek Math paper was the introduction of GRPO (Shao, 2024), a new algorithm for training LLMs with RL. Read the paper and do a literature review of subsequent methods on RLVR (Reinforcement Learning with Verifiable Rewards). Include a table of the five most relevant papers to your project (related to RLVR) with a short commentary for each.

Paper	Core Idea	How it Builds on/Differs from GRPO	Relevance to a DDI Judge
DAPO: An Open-Source LLM Reinforcement Learning System at Scale	Introduces DAPO, a scalable RL algorithm that reaches 50 on AIME 2024 with Qwen2.5-32B, and fully open-sources code, data, and recipes. The method combines four techniques - Clip-Higher, Dynamic Sampling, Token-Level Policy Gradient Loss, and Overlong Reward Shaping - to stabilize long-CoT RL and outperform prior GRPO baselines.	Identifies naive GRPO failure modes in long-CoT settings: entropy collapse, reward noise, and instability; then replaces GRPO's sample-level loss with token-level loss, decouples clipping ranges to raise the exploration ceiling, dynamically buffers filtered samples, and shapes rewards for overlong outputs. It also removes the KL penalty used in GRPO to allow larger distribution shifts during reasoning.	Uses rule-based, verifiable rewards instead of a reward model, which aligns with a DDI judge that can auto-verify cases (e.g., correctness of flagged interactions against ground truth). Token-level loss and overlong shaping curb rambling rationales and stabilize long explanations; dynamic sampling focuses training on informative, filtered cases - useful when curating hard clinical interaction scenarios.
Improving Sampling Efficiency in RLVR through Adaptive Rollout and Response Reuse	Proposes AR3PO, an RLVR method that fixes GRPO's vanishing-advantage issue using two mechanisms: adaptive rollout and response reuse. It matches or beats DAPO while cutting rollout cost by up to 4.2 times on 7B-8B models and staying competitive at 32B.	GRPO's group-normalized advantages go to zero when all responses are correct or all are wrong. AR3PO allocates extra generations only to hard prompts and reuses previously correct responses by injecting their reward signal and recomputing token probabilities under the current policy, improving sample efficiency without a value model.	RL with verifiable rewards maps to rule-checked clinical outputs, so a DDI judge can score responses against ground-truth interaction labels. Adaptive rollout focuses computation on difficult patient-drug cases, and response reuse exploits past correct judgments to maintain gradients when current samples all fail or all pass. The net effect is lower compute and steadier training for long rationales.
G²RPO-A: Guided Group Relative Policy Optimization with Adaptive Guidance	Injects ground-truth reasoning steps into rollouts for small LLMs, then adapts the amount of guidance online based on training dynamics. Shows that naive fixed guidance gives no lasting gains, while adaptive guidance improves math and code benchmarks across Qwen3, DeepSeek-Math, and DeepSeek-Coder bases.	Extends GRPO by splitting each batch into guided and unguided rollouts and using reward trends to set future guidance length. Targets GRPO's weak gains on small models and sparse rewards; includes curriculum with hard samples and task-dependent guidance ratios. Outperforms vanilla GRPO and simple guided baselines.	Maps to seeding rollouts with verified clinical reasoning steps or partial interaction rationales, then tapering guidance as the judge improves. Reduces sparse-reward issues when many cases yield identical pass/fail signals and helps small models learn longer, accurate rationales. Hard-case curriculum aligns with stressing rare but critical DDIs.

[Scaf-GRPO: Scaffolded Group Relative Policy Optimization for Enhancing LLM Reasoning](#)

Defines the “learning cliff” in RLVR where zero rewards collapse GRPO’s advantage to zero. Introduces hierarchical, minimal in-prompt hints that activate only when learning plateaus, yielding large gains on hard math tasks (e.g., a 44.3% relative pass@1 improvement on AIME24 with Qwen2.5-Math-7B over vanilla GRPO). Keeps exploration flexible and on-policy while outperforming prefix-guided baselines like LUFFY.

Adds a two-phase protocol: a guidance-exemption period to separate “pseudo-hard” from “true-hard” items, then progressive hint-guided exploration with tiered hints from abstract concepts to concrete steps. Remains on-policy GRPO but augments the rollout buffer with hinted outcomes when group rewards stall, avoiding distribution shift from teacher prefixes and the instability of hybrid SFT-RL fixes. Reports consistent gains across models and scales beyond the GRPO baseline.

Maps to sparse, verifiable clinical rewards where many prompts yield identical pass/fail signals. Use hierarchical hints as scaffolded clinical cues (e.g., abstract pharmacology principles before concrete steps) to break zero-gradient cases while preserving diverse reasoning paths. Improves reliability on rare, severe DDI edge cases without locking the judge into teacher-forced trajectories.

[Critique-GRPO: Advancing LLM Reasoning with Natural Language and Numerical Feedback](#)

Online RL that fuses natural-language critiques with scalar rewards. The policy learns from both initial answers and critique-guided refinements, with a shaping function that boosts correct, unfamiliar refinements and penalizes incorrect ones. Reports consistent pass@1 gains over GRPO across math, STEM, and general reasoning.

Starts from GRPO’s group-relative advantage but augments rollouts with critique-conditioned self-refinements, keeping exploration on-policy. Adds weighted advantages and policy shaping to counter plateaus and persistent failures observed under purely numerical feedback. Demonstrates larger improvements when critiques contain CoT detail versus binary correctness.

Swap “critiques” for rule-verifiable clinical comments: why a flagged DDI is severe, which mechanism, and what patient factors amplify risk. Use critique-guided refinements to force the judge to correct shallow or wrong rationales while still training with verifiable rewards. Shaping focuses learning on rare but high-value corrections, improving top-k alert precision for severe interactions.

2 Part Two: Metrics and Evaluations (30 marks)

Create a list of the top 5 ML metrics you care about for your project. For each metric, include:

- Why do you care about this metric
- How you will measure it (quantitative and qualitative)
- Challenges with measuring the metric at scale

Include a table with your metrics and a short commentary on each subsection

Metric	Why it matters	How it will be measured	Challenges at scale
Alert Correctness@K	Measures whether the system's top-K predicted interactions match the verified DDIs for a patient-drug context. Direct indicator of model accuracy and clinical usefulness.	Quantitative: compute $\text{Correctness@K} = (\# \text{ true positives in top K}) / K$ for each case. Qualitative: review mis-ranked alerts to categorize error type (missed vs spurious).	Need comprehensive, up-to-date DDI ground-truth tables; synonym and spelling mismatches can create false negatives (i.e. needs normalization); evaluation cost grows with dataset size.
Severity-Weighted Precision@K (SW-P@K)	Clinicians act on the top-K alerts. Severe errors are costliest.	Quantitative: for the top K alerts, give each correct alert a weight by severity (Major > Moderate > Minor > Unknown). SW-P@k equals the average of these weighted correctness values across the K alerts Qualitative: pharmacist spot-check of sampled outputs against trusted DDI sources and clinical notes	Severity scales differ across data sources; limited samples for rare or extreme severities; human audits needed for ambiguous cases.
Patient-Context Adherence	DDIs vary by age, comorbidities, renal function, etc. The model must adapt its reasoning to individual patient profiles.	Quantitative: rule-pass rate = % of predictions satisfying context-specific guideline checks (e.g., dose * eGFR limits). Qualitative: manual audit of reasoning explanations to confirm context mention.	Requires codified clinical rules and accurate patient metadata; context coverage is uneven across datasets; rule engine maintenance overhead.
Justification and Evidence Quality	Builds clinician trust. Alerts must cite valid mechanisms or references, not hallucinated facts.	Quantitative: fraction of outputs containing a verified drug pair + known mechanism + trusted source link (check via regex + reference DB). Qualitative: 3-point rubric (accurate/partial/hallucinated).	Citation validation at scale requires an automated parser and a continuously updated whitelist of trusted sources; hallucination detection is imperfect.
Hallucination and Unsafe-Content Rate	Ensures safety by penalizing fabricated or medically impossible statements.	Quantitative: % of outputs flagged by named-entity checks against a curated drug dictionary and forbidden-term list. Qualitative: review of false-positive flags to refine rules.	Drug vocabularies evolve; high recall rules may over-flag valid novel drugs; maintaining the dictionary and verifying edge cases is labour-intensive.

3 Part Three: Establishing a baseline with an LLM (15 marks)

Pick one of your metrics and create a basic classifier using one LLM with prompting and a simple baseline (e.g. logistic regression on embeddings). Build a training, validation and evaluation dataset, compare the results, discuss the limitations and next steps for the classifier. Include a breakdown of common mistakes with qualitative and quantitative analysis.

Logistic Model Code:

<https://colab.research.google.com/drive/1TNFic-9mCh71ESI4GroNkYazdb4iNNDI#scrollTo=jj2s19KSVhcJ>

Baseline Model:

Task and Evaluation Metric: The objective is to predict the severity level (Minor, Moderate, Major) of a drug–drug interaction (DDI) given structured patient and exposure information. Because the dataset is highly imbalanced (Minor=116, Moderate=2398, Major=1684), we evaluate all models using Macro F1, which weights each class equally regardless of frequency.

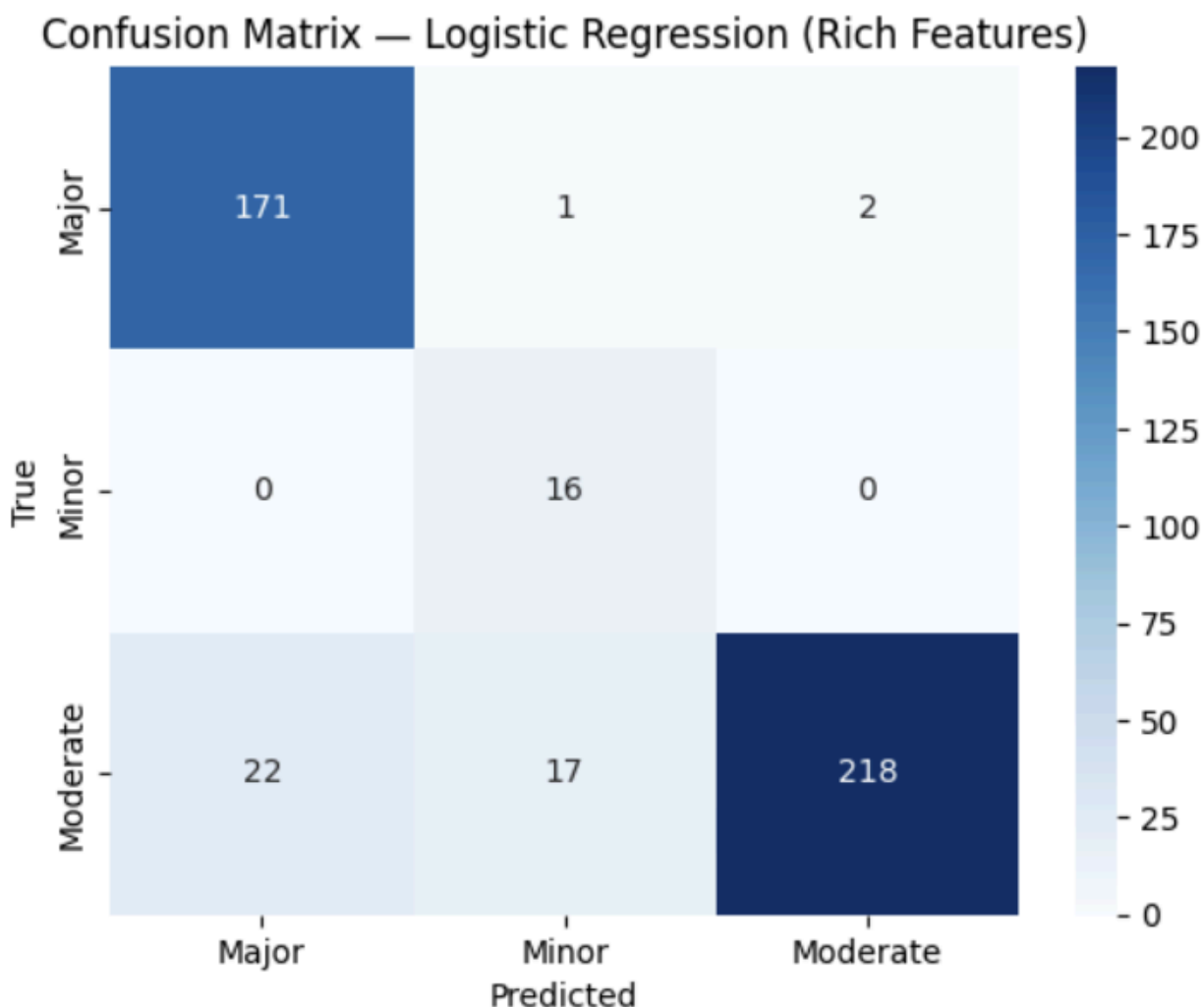
Baseline Model (Logistic Regression): We developed a multinomial logistic regression classifier as the baseline model. After cleaning and deduplicating 10 000+ interaction records, we restricted analysis to entries with valid unified severity labels and decided on features: age, sex, comorbidity count, overlap duration, DDI confidence (number of sources), DDI known (boolean), and TF-IDF representation (3 000 terms) of the unified_mechanism_text describing each interaction's pharmacological mechanism. Duplicates were removed prior to splitting to prevent information leakage.

Results: The final baseline was trained and evaluated using a random row level split (overlapping drug pairs allowed), reflecting the practical goal of classifying known DDIs under new patient contexts. Under this setup, the model achieved the below results:

Severity	Precision	Recall	F1	Support
Major	0.9	0.98	0.94	174
Moderate	0.99	0.86	0.92	257
Minor	0.47	1.00	0.64	16

Overall, this model achieved 83% Macro F1 and 91% overall accuracy. The model performs strongly on Major and Moderate interactions, correctly identifying most high severity cases (recall > 0.95 for Major). For the Minor class, recall reaches 1.0 but precision drops to 0.47, which is likely due to the lack of data samples.

The below confusion matrix indicates that the model accurately classifies the majority of Major and Moderate DDIs, though it shows a mild tendency to overestimate severity. All Minor interactions are correctly identified, but a portion of Moderate cases are misclassified as Minor, reducing precision for that class. Overall, the results reflect strong performance and a clear safety oriented bias, where the model prefers to err toward predicting higher severity rather than risk overlooking serious interactions.



When evaluated on unseen drug pairs (pair-held-out split), the model's Macro F1 dropped to approximately 0.18.

This outcome indicates that while the logistic regression baseline effectively predicts severity for known drug combinations using contextual and mechanistic information (mechanism text, comorbidity burden, DDI confidence etc), these features alone were not sufficient to generalize to entirely new pairs. In other words, the model's understanding of severity remains closely tied to the specific drugs seen during training rather than to broader pharmacological patterns. This motivates the use of a language-based LLM classifier, which can interpret mechanistic text more

flexibly and infer the severity of novel drug combinations by reasoning about their shared mechanisms/effects.

LLM Classifier - Code on Github under folder llm_eval

To compare against the logistic regression baseline, we evaluated a prompt based LLM classifier using the same dataset split. Each test example was formatted as a structured prompt containing all the information in each row of our dataset. We used AWS's Amazon Bedrock model service along with the openai.gpt-oss-120b-1:0 model for this evaluation.

Example Prompt:

"""

A patient is concurrently prescribed two medications.

Patient information:

- Age: 72
- Sex: Female
- Comorbidities: Hypertension, Type 2 diabetes, Chronic kidney disease

Medication exposure:

- Start: 2024-06-15
- End: 2024-08-02
- Overlap days: 48

Drugs:

- Drug 1: Verapamil
- Drug 2: Digoxin
- Known interaction in clinical sources: True
- Number of data sources supporting this: 5

Mechanistic context:

Verapamil inhibits P-glycoprotein-mediated efflux and CYP3A4 metabolism of digoxin, markedly reducing digoxin clearance and increasing plasma concentrations, which can lead to bradycardia and atrioventricular block.

Return:

- 1) A JSON object: {"severity": "<Minor|Moderate|Major>"} (exactly).
- 2) 1–3 sentence explanation citing your decision

"""

Example Output:

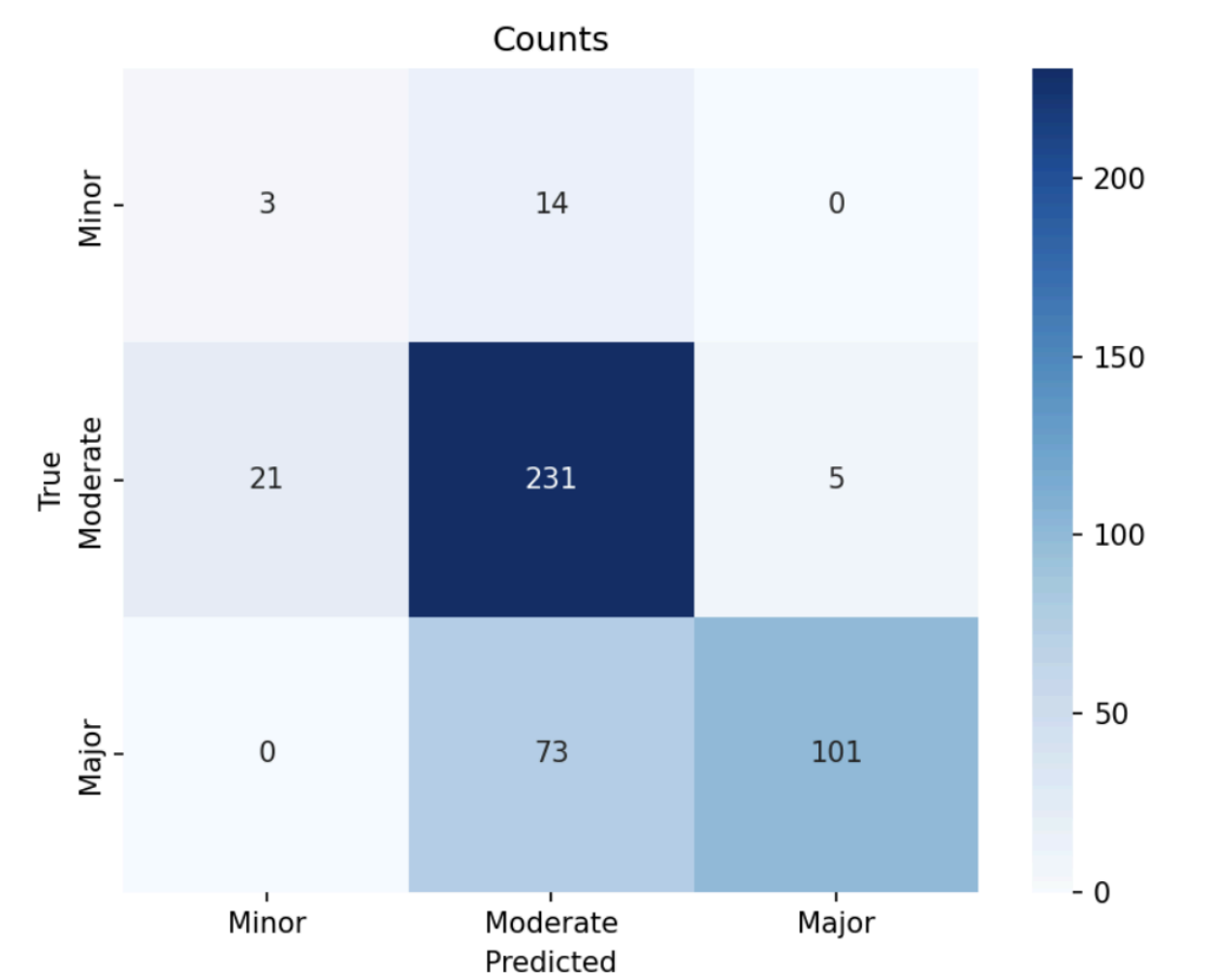
{"severity": "Major"}

This interaction is classified as ****major**** because verapamil significantly inhibits

the elimination of digoxin via both P-glycoprotein and CYP3A4 pathways, leading to potentially toxic plasma levels. The resulting risk of arrhythmia and heart block warrants close monitoring or avoidance of the combination.

Results: The LLM classification achieved an overall accuracy of 75% along with an overall Macro F1 of 56%. The per class breakdown and confusion matrix are as follows:

Severity	Precision	Recall	F1	Support
Minor	0.13	0.18	0.15	17
Moderate	0.73	0.9	0.8	257
Major	0.95	0.58	0.72	174



Evaluating the LLM on purely few-shot classification underperformed the logistic regression baseline. Performance was dominated by the Moderate class (F1=0.80), while predictions for Minor and Major severities were notably less consistent. The model displayed a strong bias toward Moderate severity, with 73 Major -> Moderate and 14 Minor -> Moderate misclassifications accounting for most major errors. This pattern shows that it appears the model defaults to “Moderate” when uncertain. However, safety appeared to be preserved as the model maintained a reasonably high recall for critical interactions (Major=0.58 vs. Minor=0.18), rarely missing truly dangerous DDIs.

For further improvements, we plan on experimenting with few-shot classification, some sort of hybrid approach with combining the logistic regression + LLM classifications, and fine tuning which is explored in part 4.

4 Part Four: Training your Judge (35 marks)

Design and implement a RLVR training pipeline for your project for one of your metrics. Create a report with:

- The metric you are measuring and the hypotheses on training success
- Your training pipeline
- The base model you are fine-tuning
- The RL algorithm you are using and why
- Logging and metrics of the training runs (you can use a commercial or open source tool like TensorBoard, wandb, etc.)
- Ablations you ran with hyperparameters
- What didn't work and next steps.

Make sure to include your code in your PR

1. Metric and Hypotheses

Metric:

The primary metric for evaluating the model is Severity-Weighted Precision at K (SW-P@k):

- Definition: SW-P@k measures the weighted correctness of the top K predictions, where weights are assigned based on severity levels:
 - Major: 3.0
 - Moderate: 2.0
 - Minor: 1.0
 - Unknown: 0.0

- Formula:
$$\text{SW-P@k} = \frac{\sum_{i=1}^k \text{weight}(\text{correctness})}{k}$$

Hypotheses:

1. Reward Signal: A reward function based on severity prediction accuracy will guide the model to prioritize high-severity predictions (e.g., "Major") over lower-severity ones.
2. Fine-Tuning Impact: Fine-tuning the base model with RL will improve SW-P@k and overall accuracy compared to the pre-trained model.
3. Hyperparameter Sensitivity: Learning rate and batch size will significantly

2. Training Pipeline

Steps:

1. Dataset Preparation:
 - Input: JSON dataset with prompts and completions.
 - Output: Processed dataset with query, reference, and actual_severity fields.
2. Model Initialization:
 - Base model: Qwen/Qwen2.5-0.5B (a causal language model).
 - Tokenizer: Configured with padding and special tokens.
3. Reward Function:
 - Extracts predicted severity from model completions.
 - Compares predictions to ground truth (actual_severity) and assigns rewards based on correctness and severity weights.
4. Training:
 - Algorithm: Group Relative Policy Optimization (GRPO).
 - Objective: Maximize rewards using the custom reward function.
 - Logging: TensorBoard tracks loss, rewards, and learning rate.
5. Evaluation:
 - Metrics: SW-P@k, overall accuracy, and per-severity accuracy.
 - Outputs: Predictions, rewards, and evaluation metrics.
6. Logging and Visualization:
 - TensorBoard for real-time monitoring.
 - Training metrics saved as JSON and visualized with Matplotlib.

3. Base Model

Model: Qwen/Qwen2.5-0.5B

- Type: Causal language model.
- Pre-training: Trained on diverse datasets for general-purpose language understanding.
- Reason for Selection: Strong performance on text generation tasks and compatibility with Hugging Face's Transformers library.

4. RL Algorithm

Algorithm: Group Relative Policy Optimization (GRPO)

- Why GRPO?
 - A recent paper, Sheng Z. et al. Med-RLVR tested GRPO with Qwen-3B on the Medical tests dataset and found it performing well.
 - GRPO is designed for fine-tuning language models with reinforcement learning.
 - It balances exploration (generating diverse completions) and exploitation (optimizing for rewards).
 - KL divergence regularization ensures the fine-tuned model doesn't deviate too far from the base model, maintaining fluency and coherence.

5. Logging and Metrics

Tools:

- TensorBoard:
 - Tracks training loss, rewards, and learning rate schedules.
 - Visualizes metrics in real-time.
- Custom Metrics:
 - SW-P@k and per-severity accuracy are logged after training.
 - Training history and evaluation results are saved as JSON for further analysis.

6. Ablations

Hyperparameters Tested:

1. Learning Rate:
 - Tested values: ($1e{-5}$, $5e{-6}$, $1e{-6}$).
 - Observation: ($5e{-6}$) provided the best balance between convergence speed and stability.
2. Tried comparing the GRPO with [Qwen-0.6B finetuned with SFT](#)

7. Next Steps

- Analyze results by reviewing training metrics, and logs
- Deploy the Model
 - Save Final Model:
 - Use the best-performing checkpoint for deployment.
 - Integrate with Application:
 - Deploy the model to predict DDI severity in real-world scenarios

5 Submission Instructions

Submit a pull request to the course GitHub repo with your assignment in a folder named a4 with your a4.pdf on a branch called a4. Include your team members' names on the first page with student IDs. Also, upload your assignment to Quercus.